



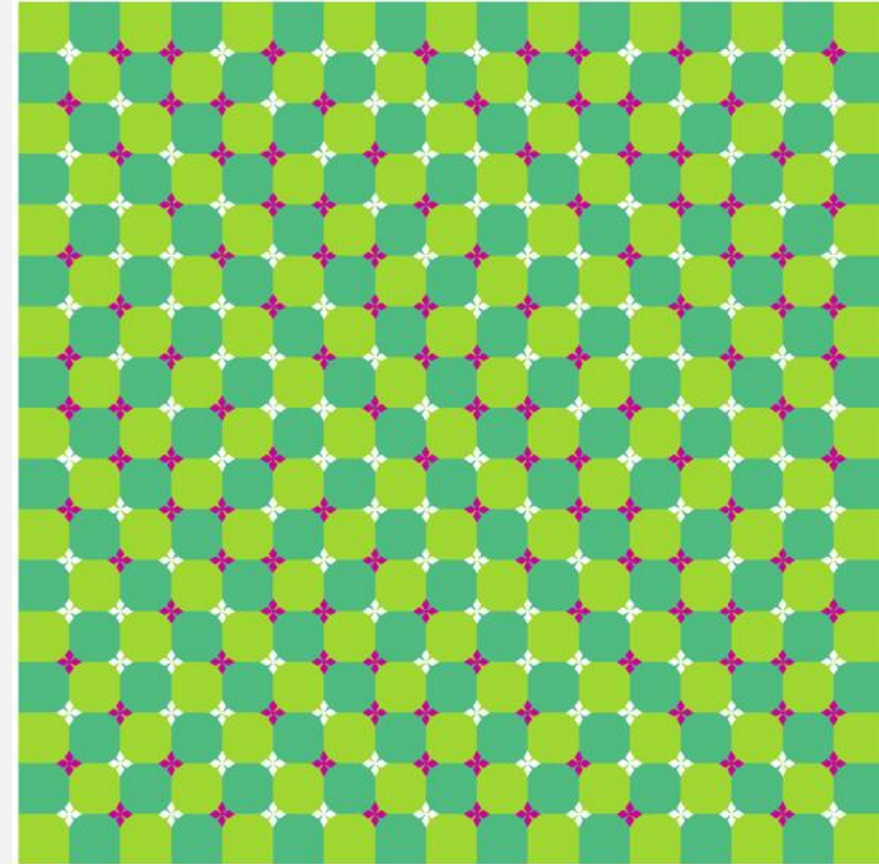
CS 1910

Computer Science I
Variables, Literals and
Types

TRY AT HOME RECAP

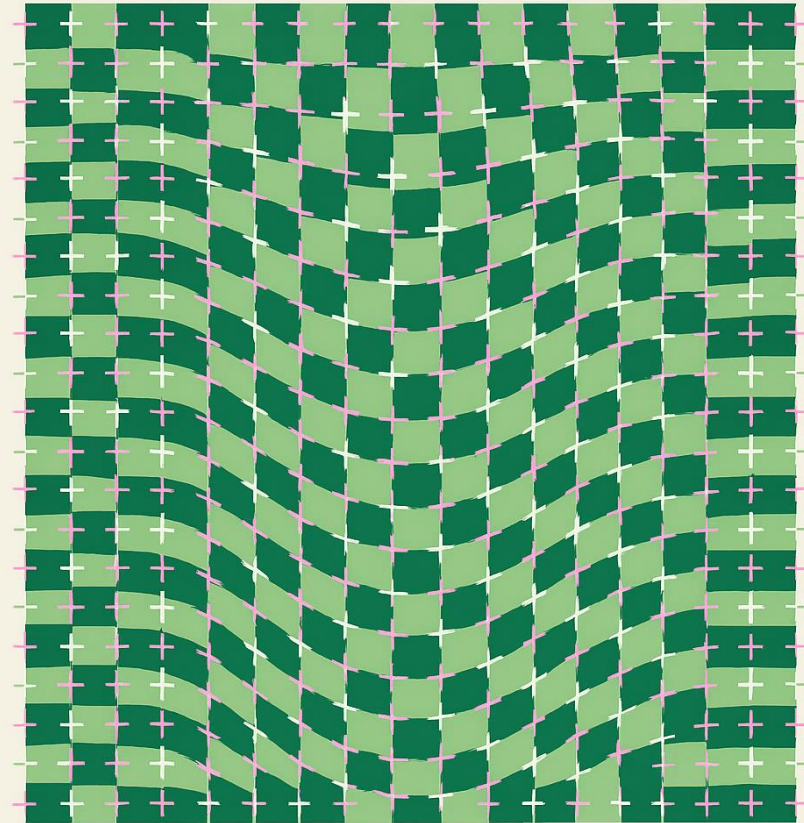
- Turtle Draw – initial in name
- Tinkering with the prior code – any interesting outcomes?
- GenAI for the wavey illusion
 - Success or fail?
 - What was your strategy?
 - How many iterations?
 - How did you debug?

Find errors/shortcomings and fix them?



MY 1ST ATTEMPT

- Prompt:
 - I want you to generate the code for an optical illusion. The illusion is a wavy grid of boxes alternating with a light green and a dark green. There are 17 small squares along both sides of the square grid. At each intersection between the squares is a small cross along the gridlines with the cross extending about 1/6th across the line into the square. The crosses are either pink or white. the first intersection of grid lines has a white cross. Then there is two pink crosses. Then a single white cross. then a single pink cross. Then there are two white crosses. then a pink cross. then a white cross. Then two pink crosses. and then that pattern repeats until you are at the end of the intersection points with a pink cross. On the second line there is a Pink, white, 2 pink, white, pink, two white and so on. This pattern then repeats. Please generate the picture for me, and then python code to generate it.
- Blue is the *context* provided to the AI, while the red is the task I am giving it to do.

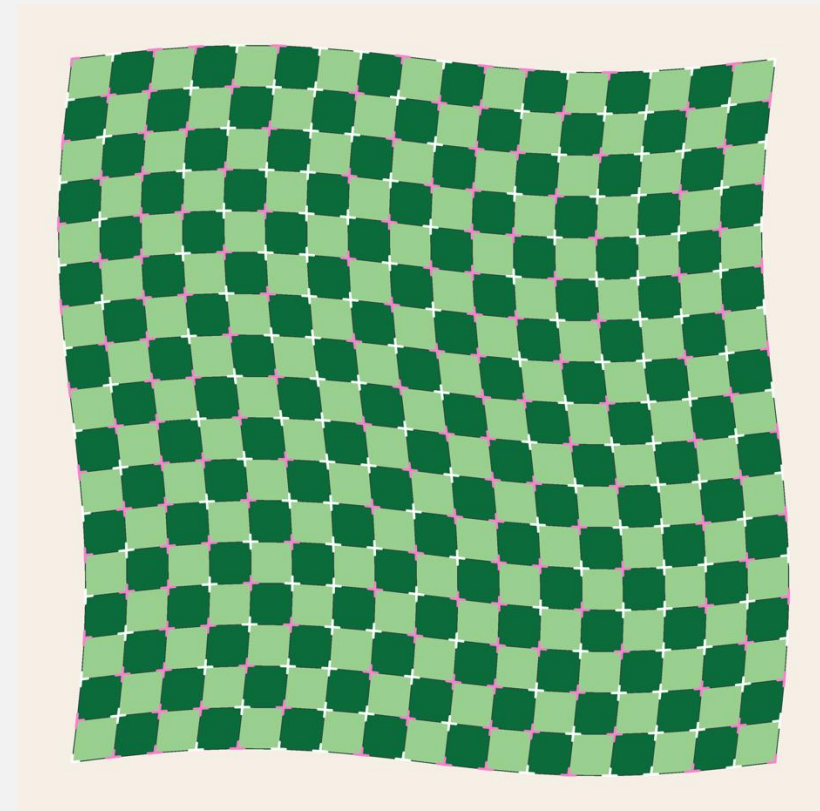


MY 2ND ATTEMPT

- Prompt:

Some of your crosses are not at the intersections of the gridlines, but are instead in the middle of the lines. Make sure that the centre of the cross is at the intersection, with the perpendicular pieces of the cross going exactly along the gridlines. I want you to generate the code for an optical illusion. The illusion is a wavy grid of boxes alternating with a light green and a dark green. There are 17 small squares along both sides of the square grid. At each intersection between the squares is a small cross along the gridlines with the cross extending about 1/6th across the line into the square. The crosses are either pink or white. the first intersection of grid lines has a white cross. Then there is two pink crosses. Then a single white cross. then a single pink cross. Then there are two white crosses. then a pink cross. then a white cross. Then two pink crosses. and then that pattern repeats until you are at the end of the intersection points with a pink cross. On the second line there is a Pink, white, 2 pink, white, pink, two white and so on. This pattern then repeats. **Please generate the picture for me, and then python code to generate it. Check to make sure that all of the crosses are actually on the intersections of the gridlines. Blue is the context provided to the AI, while the red is the task I am giving it to do.**

- Purple is feedback based on the first attempt. That will add it to the context that the AI will refer to. Then the instructions again, then the task, and then a set of validation instructions to have it check its work. It gets a lot closer!



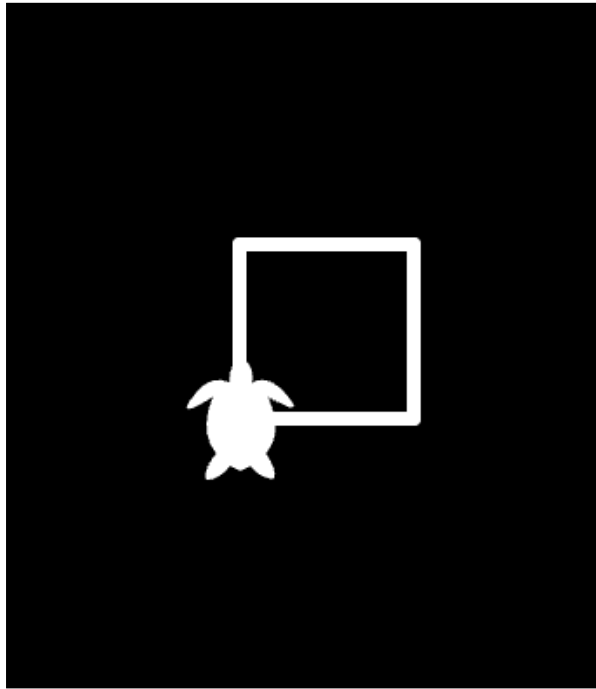
PSEUDO CODE (REVISITED)

- Outline the logic and structure using plain language (with some structure)
 - Supports programming constructs
 - Blocks
 - Repetition
 - Decisions
 - Variables
- Code that belongs to a specific scope
- Ability to repeat certain instructions for a specified duration
- Code that is executed only when criteria are met
- Save data to memory, abstract programs for readability
-
- The diagram consists of several orange arrows pointing from descriptive text on the right to specific items in the list on the left. One arrow points from 'Code that belongs to a specific scope' to 'Blocks'. Another arrow points from 'Ability to repeat certain instructions for a specified duration' to 'Repetition'. A third arrow points from 'Code that is executed only when criteria are met' to 'Decisions'. A fourth arrow points from 'Save data to memory, abstract programs for readability' to 'Variables'.

SQUARE CODE

Consider we want to apply computational thinking to the draw a square task from last class.

```
5 #Draw a square
6
7 #draw the 4 sides
8 forward(50)
9 right(90)
10
11 forward(50)
12 right(90)
13
14 forward(50)
15 right(90)
16
17 forward(50)
18 right(90)
```



Decomposition:

- draw straight lines with specific length
- rotate

Patterns:

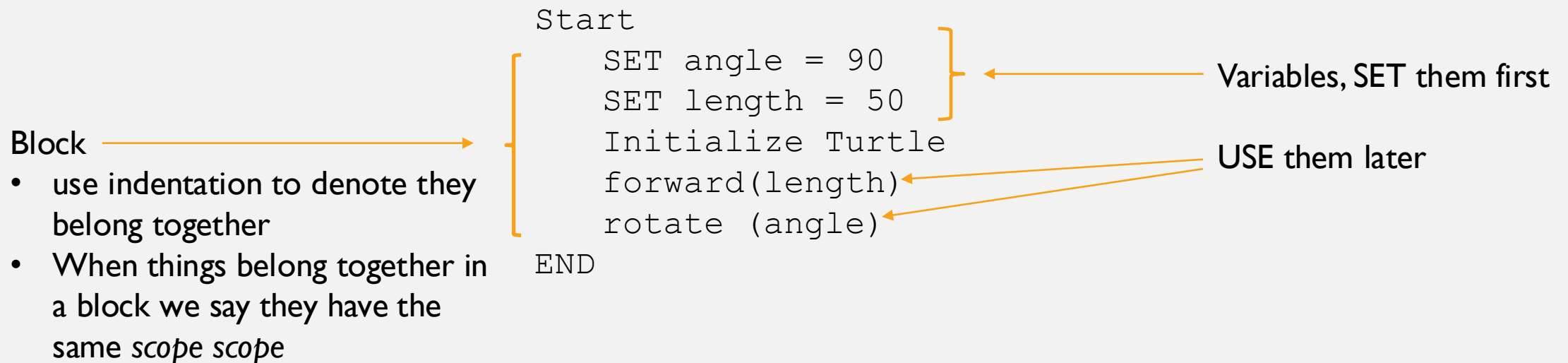
- 4 sides with exact same characteristics

Abstraction:

- Right now we have very specific values for how far to draw and turn.
- If we have to change those values, we have to change them in many places.
- We can **abstract** those values:
 - Draw – it goes a **length** of a line (50)
 - Turn – it rotates a particular **angle** (90)
- Because it is always that we need a number that we reuse, with a name, we can create a *variable* to store the value and reuse it
- Is there anything else we might do this with?

REFINING PSEUDOCODE (DRAW A SQUARE)

- Even though we have some python code to draw a square – we can go back to pseudocode and *iterate*
 - Iteration – repeating problem solving to improve some quality of our solution
- Start with draw a line and rotate (that's a basic decomposed aspect of drawing a square)
- Use variables to represent specific values (a basic abstraction of the shape – side length and rotation angle)
- Program to draw a line with specific length and rotate a specific angle:



PSEUDOCODE (DRAW A SQUARE)

Note:

- We DO NOT need to repeat setting up the variables 4 times
- Each line and rotate uses the same values
- We really only need to initialize Turtle once
- Doing these again and again is *inefficient* – if we can save steps that is important in computing

Draw a Square
Do this 4 times ...

Start

SET angle = 90
SET length = 50
Initialize Turtle
forward(length)
rotate (angle)

END

Draw a line and rotate
For a square we repeat this
4 times

PSEUDOCODE (DRAW A SQUARE)

Start

SET angle = 90

SET length = 50

Initialize Turtle

FOR i = 1 to 4:
 forward(length)
 rotate (angle)

END

New Block (indentation)
Indicates this code belongs to the
Repetition

Repetition
Do the block that follows
for i=1,2,3,4

LOOP UNROLL

Programs that use repetition can be “unrolled” into sequential form

Start

```
SET angle = 90  
SET length = 50  
Initialize Turtle
```

```
FOR i = 1 to 4:  
    forward(length)  
    rotate (angle)
```

END

Equivalent programs



Start

```
SET angle = 90  
SET length = 50  
Initialize Turtle
```

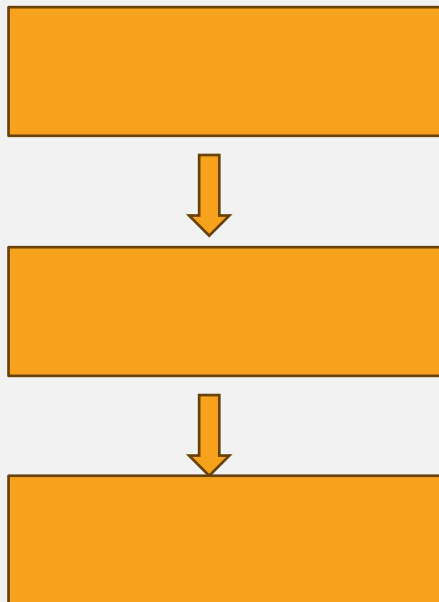
```
i = 1  
forward(length)  
rotate (angle)  
i = 2  
forward(length)  
rotate (angle)  
i = 3  
forward(length)  
rotate (angle)  
i = 4  
forward(length)  
rotate (angle)
```

END

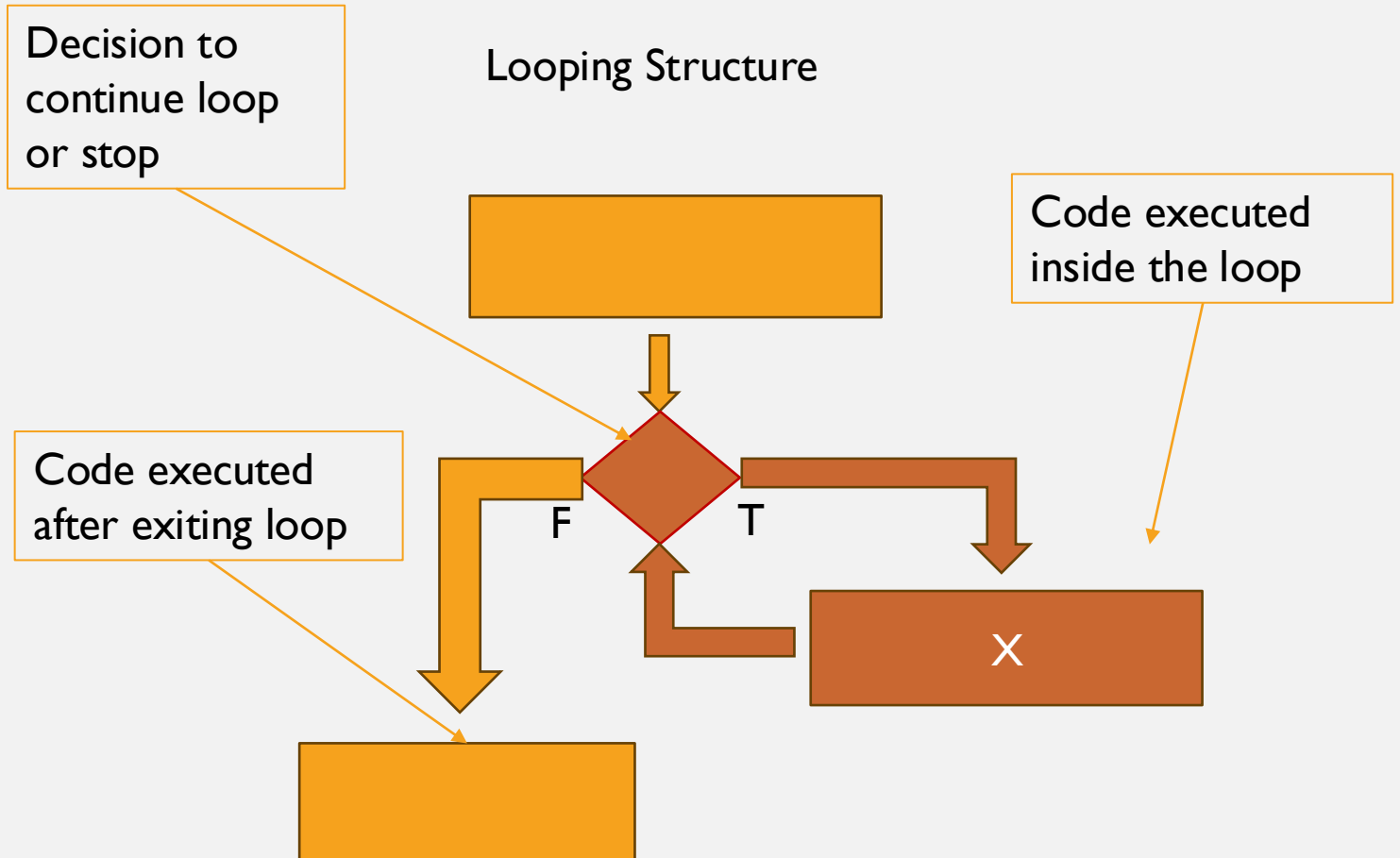
Note the indent –
only stuff under
FOR is in scope for
the repetition

PROGRAM FLOW

Sequential Program Flow



Looping Structure



PSEUDOCODE TO PYTHON

Pseudocode

Start

SET angle = 90

SET length = 50

Initialize Turtle

FOR i = 1 to 4:

 forward(length)

 rotate (angle)

END

Python

```
from ColabTurtle.Turtle import *
```

```
angle = 90
```

```
length = 50
```

```
initializeTurtle()
```

```
for i in range(4):
```

```
    forward(length)
```

```
    right(angle)
```

← Python variables

← Actually goes from

i = 0, 1, 2, 3

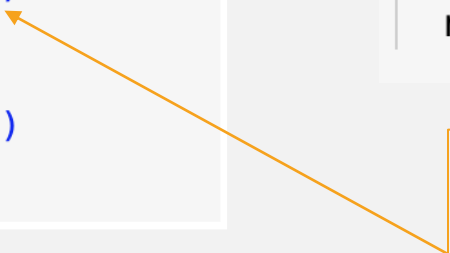
Up to but not including the
number specified

← Python for loop

Programming nuance: counting often starts at zero and often goes up to but not including the end

COMPARISON

```
5 #Draw a square
6
7 #draw the 4 sides
8 forward(50)
9 right(90)
10
11 forward(50)
12 right(90)
13
14 forward(50)
15 right(90)
16
17 forward(50)
18 right(90)
```



```
#draw a square

angle = 90
length = 50
initializeTurtle()

for i in range(4):
    #draw a line and rotate
    forward(length)
    right(angle)
```

Consider we want to adjust the size of the square. In our second program we only update 1 variable as opposed to 4 different lines of code

↑ readability
↑ maintenance
↑ conciseness
↑ generalization

- Values typed directly into a program (eg., the number 50) are called ***literals***, we also say they are ***hardcoded*** values
- In general, we do not want hardcoded values because it is hard to maintain.

FIND A PARTNER

- What if we wanted to draw a triangle?
- What if we wanted to draw a different regular polygon?

Closed shape with all angles and side lengths equal
eg., pentagon, hexagon, dodecagon, square

Task 1: Give pseudocode to draw a triangle

- First question – what do you have that you can reuse?
- Second question – what do you need to solve the problem?
 - How many sides are in a triangle?
 - What is the interior angle for an equilateral triangle?

Task 2:

- Give pseudocode to draw any regular polygon
 - Hint: Revisit the idea of abstracting the shape
- Introduce a new variable called sides
- If sides is 6 your code should draw a hexagon
- If you finish early, reuse the code in previous slides to try to convert your pseudocode to python

DRAWING SHAPES

Triangle

Start

```
SET angle = 120  
SET length = 50  
Initialize Turtle
```

```
FOR i = 1 to 3:  
    forward(length)  
    rotate (angle)
```

END

Regular Polygon

Start

```
SET sides = 5  
SET angle = 360 / sides  
SET length = 50  
Initialize Turtle
```

```
FOR i = 1 to sides:  
    forward(length)  
    rotate (angle)
```

END

sides



PYTHON POLYGON

Start

```
SET sides = 5
SET angle = 360 / sides
SET length = 40
Initialize Turtle

FOR i = 1 to sides:
    forward(length)
    rotate (angle)
```

END

```
from ColabTurtle.Turtle import *
```

```
#draw a regular polygon
```

```
sides = 5
angle = 360 / sides
length = 40
initializeTurtle()
```

```
for i in range(sides):
    #draw a line and rotate
    forward(length)
    right(angle)
```

VARIABLES

- Variable: Symbolic names that refer to objects or values stored in memory of a computer

- Example:

- `length = 50`

- Assignment *operator* – means “assign length value 50” not equals

- **length** is the name of a variable referencing a place in computer memory where a programmer can find a value (hiding detail of the memory - abstraction)

- Value that length ‘gets’ as its value to store. Now whenever a programmer types the variable length it will return 50 from memory to use
- 50 is a *literal* that is assigned to the *variable*

Good Programmer Habit:

- Read the assignment operator as “GETS”
- The above python line reads as: length “gets” 50

VARIABLES

- Must be assigned (appear on the left side of the assignment operator (=) **before** they are used)
- Can be reassigned new values
- Must follow specific naming conventions
- Have a defined **scope**
 - Remember our indentation – it works in Python too
 - A variable is defined in the block (indented region of code) and are only accessible from there
 - **Advanced concept:** Scopes can be **nested**. Assume Block A defines a variable length, and a Block B nested under it. Block B can also see that variable. But after B ends we lose all of its variables that it defines. (More on this later)

NAMING CONVENTIONS

- Must start with a *letter* or an *underscore* (_ - usually available by holding shift + minus key)
 - Cannot start with a number or any other symbol
 - Can only contain alphanumeric characters (A-z, 0-9) and underscores
- Are case sensitive (length is different from Length or LENGTH)
- Cannot be a reserved keyword – what's that?
 - A keyword that is already used by Python in its definition

PYTHON KEYWORDS

Value Keywords	<code>True</code> , <code>False</code> , <code>None</code>
Operator Keywords	<code>and</code> , <code>or</code> , <code>not</code> , <code>is</code> , <code>in</code>
Control Flow Keywords	<code>if</code> , <code>else</code> , <code>elif</code> , <code>for</code> , <code>while</code> , <code>break</code> , <code>continue</code> , <code>pass</code>
Function and Class Definition	<code>def</code> , <code>return</code> , <code>lambda</code> , <code>yield</code> , <code>class</code>
Exception Handling	<code>try</code> , <code>except</code> , <code>finally</code> , <code>raise</code> , <code>assert</code>
Module and Import	<code>from</code> , <code>import</code>
Scope Management	<code>global</code> , <code>nonlocal</code>
Context Management	<code>with</code> , <code>as</code>
Asynchronous Programming	<code>async</code> , <code>await</code>
Deletion	<code>del</code>

PYTHON VARIABLE NAME CONVENTIONS AND BEST PRACTICES

- **Readability:** Names should be clear, descriptive, and easy to understand.
 - Example: **salary** instead of **x**
 - One exception – programmers will often use single letter variables for counting things in repetitions (remember our FOR – it used the letter **i**)
- **Consistency:** Maintain a consistent naming style throughout your codebase.
 - Typically lowercase with words separated by underscores
 - Example: **first_name**
- **Avoid ambiguity:** Do not use names that could be easily confused with other variables, functions, or keywords, and may cause errors.
 - Try to avoid things like: **int**, **max**, **abs** (all built in function names)

Our python code to draw a square could use this



MORE NAMING CONVENTIONS

- Use a trailing underscore to avoid collisions with keywords or builtins
 - eg. `max_`
- Single underscore (by itself) can be used to signify a throw-away variable
 - E.g., `for _ in range(4): #no plans to use _ in the loop`
- **ALL_CAPS** with underscores denotes **CONSTANTS** (variables that are stored in memory but shouldn't be modified by programmers)
 - E.g., `KGS_TO_LBS = 2.20462`
- Single leading underscore indicates a private variable (covered in CS 1920)

SELF-CHECK

- Which of the following are valid python variable names?

int	my-name	local	Try
NAME	Long^Game	Global	False
3xtra	_view	LongGame	num_of_visitors
P	P0t4t0	4ever	try_again

- Which of them are poor choices and why?

SELF CHECK

What is a variable in python?

- A. a reserved word
- B. a data type
- C. a location in memory to store data
- D. a function

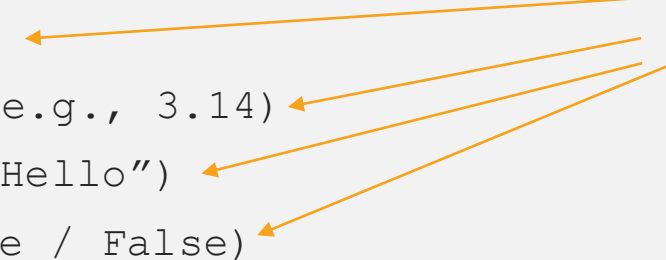
CLASS PARTICIPATION PAUSE

- See the link on Moodle!

PYTHON DATA TYPES

- Allow you to represent different types of data
- Basic data types:
 - int = integer numbers (e.g., 10)
 - float = floating point numbers (e.g., 3.14)
 - str = character **strings** (e.g., "Hello")
 - bool = boolean values (e.g., True / False)
 - None = a special type that represents the absence of a value (more on this later)
- Variables have an underlying type – you can find out what type a variable is by using the following: `type(name_of_variable)`
 - Advanced Concept: `type(...)` is a built-in function in Python
- There are less common types (e.g complex, bytes, bytearray)

Examples of literals



INT

- Integers are whole numbers
 - Includes positive and negative numbers
 - Python has no maximum or minimum integer
 - Built-in function `int()` tries to convert other types to int values
 - `int("1"), int(1.75), int(True), int(False)`

```
my_int = 42  
type(my_int)
```

Returns class 'int'



Can be used in Expressions: `other_int = my_int * 10 + 2`

FLOAT

- Float represent floating point numbers
 - Includes positive and negative numbers
 - `float('inf')` represents positive infinity
 - Built-in function `float()` tries to convert other types to float values
 - `float("1")`, `float(1)`, `float("1.75")`,
`float(True)`, `float(False)`

```
my_float= 42.0  
type(my_float)
```

← class 'float'

Can be used in expressions: e.g., `my_float = my_float * 2.7`

STRINGS

```
my_string = "HELLO"
```

Character	H	E	L	L	O
Index Position	0	1	2	3	4

- Represents character strings
- String literals are enclosed in double `" "` or single `' '` quotes
- Individual characters in a string can be referenced using square brackets `[ ]`
 - `"Hello"[2]` ← `"L"`
- Built-in function `len( )` can be used to determine the length of a string
 - Example: `len("hello")` gives 5
- `""` represents the empty string, it has length 0
- Strings have type `str`
- `str( )` can be used to convert other types into strings

BOOL

- Short for boolean (named after English Mathematician George Boole)
- Represent `True` or `False` values
 - False is typically represented in memory by 0,
 - True typically represented in memory 1
- Control program flow
 - Allows for computers to make decisions
 - If true, then do something,
 - Else, then it is false, so do something else

LITERALS

- In programming literals are values that are directly represented within the source
 - `50` means the number (integer) `50`
 - `"Hello"` is a string literal
 - `4.2` is a floating point literal because it has a decimal
 - `True` `False` are both Boolean literals and reserved keywords
 - `Hello` (without the quotes) is not a literal, the python interpreter will see this as a variable name

VALUE ERRORS

- You can convert between types using
 - `int()`
 - `float()`
 - `bool()`
 - `str()`
- If you pass a value to those functions that cannot be converted, then it will create a Value Error
- Don't panic – you just need to work out what type of value you need

```
3 int("hello")
```

```
-----  
ValueError                                Traceback (most recent call last)  
/tmp/ipython-input-14-3011022582.py in <cell line: 0>()  
      1 #examples of int, float, bool, and str  
      2  
----> 3 int("hello")  
  
ValueError: invalid literal for int() with base 10: 'hello'
```

TYPE TEST

- In the accompanying code examples try the type conversion test
 - https://colab.research.google.com/github/UPEI-CS/CS1910-Students/blob/main/lectures/03-Variables-Literals-Types/CS1910_Variables-Literals-Types_Workbook.ipynb
- Guess the result and the uncomment and run each line individually (and then re-comment). See how many you get correct.

TRY AT HOME

- Complete the turtle program from last day to draw the Hering illusion
 - Increase the number of spokes from 4 to some number such that the illusion becomes visible
 - Use variables and loops using models from this slide deck
- Alter the turtle program that draws a polygon to use a variable named `_` in the for loop instead of `i`
- Given code to draw a letter E, alter the code to use variables to make it more readable (see Colab examples)